
17

BOUNDARIES: DRAWING LINES



Software architecture is the art of drawing lines that I call *boundaries*. Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other. Some of those lines are drawn very early in a project's life—even before any code is written. Others are drawn much later. Those that are drawn early are drawn for the purposes of deferring decisions for as long as possible, and of keeping those decisions from polluting the core business logic.

Recall that the goal of an architect is to minimize the human resources required to build and maintain the required system. What it is that saps this kind of people-power? *Coupling*—and especially coupling to premature decisions.

Which kinds of decisions are premature? Decisions that have nothing to do with the

business requirements—the use cases—of the system. These include decisions about frameworks, databases, web servers, utility libraries, dependency injection, and the like. A good system architecture is one in which decisions like these are rendered ancillary and deferrable. A good system architecture does not depend on those decisions. A good system architecture allows those decisions to be made at the latest possible moment, without significant impact.

A COUPLE OF SAD STORIES

Here's the sad story of company P, which serves as a warning about making premature decisions. In the 1980s the founders of P wrote a simple monolithic desktop application. They enjoyed a great deal of success and grew the product through the 1990s into a popular and successful desktop GUI application.

But then, in the late 1990s, the web emerged as a force. Suddenly everybody had to have a web solution, and P was no exception. P's customers clamored for a version of the product on the web. To meet this demand, the company hired a bunch of hotshot twenty-something Java programmers and embarked upon a project to webify their product.

The Java guys had dreams of server farms dancing in their heads, so they adopted a rich three-tiered “architecture”¹ that they could distribute through such farms. There would be servers for the GUI, servers for the middleware, and servers for the database. Of course.

The programmers decided, very early on, that all domain objects would have three instantiations: one in the GUI tier, one in the middleware tier, and one in the database tier. Since these instantiations lived on different machines, a rich system of interprocessor and inter-tier communications was set up. Method invocations between tiers were converted to objects, serialized, and marshaled across the wire.

Now imagine what it took to implement a simple feature like adding a new field to an existing record. That field had to be added to the classes in all three tiers, and to several of the inter-tier messages. Since data traveled in both directions, four message protocols needed to be designed. Each protocol had a sending and receiving side, so eight protocol handlers were required. Three executables had to be built, each with three updated business objects, four new messages, and eight new handlers.

And think of what those executables had to do to implement the simplest of features. Think of all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, timeout managers, retry scenarios, and all the other extra stuff that you have to do just to get one simple thing done.

Of course, during development the programmers did not have a server farm. Indeed, they simply ran all three executables in three different processes on a single machine. They developed this way for several years. But they were convinced that their architecture was right. And so, even though they were executing in a single machine, they continued all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, and all the extra stuff in a single machine.

The irony is that company P never sold a system that required a server farm. Every system they ever deployed was a single server. And in that single server all three executables continued all the object instantiations, all the serializations, all the marshaling and de-marshaling, all the building and parsing of messages, all the socket communications, and all the extra stuff, in anticipation of a server farm that never existed, and never would.

The tragedy is that the architects, by making a premature decision, multiplied the development effort enormously.

The story of P is not isolated. I've seen it many times and in many places. Indeed, P is a superposition of all those places.

But there are worse fates than P.

Consider W, a local business that manages fleets of company cars. They recently hired an "Architect" to get their rag-tag software effort under control. And, let me tell you, control was this guy's middle name. He quickly realized that what this little operation needed was a full-blown, *enterprise-scale*, **service-oriented** "ARCHITECTURE." He created a huge domain model of all the different "objects" in the business, designed a suite of services to manage these domain objects, and put all the developers on a path to *Hell*. As a simple example, suppose you wanted to add the name, address, and phone number of a contact person to a sales record. You had to go to the `ServiceRegistry` and ask for the service ID of the `ContactService`. Then you had to send a `CreateContact` message to the `ContactService`. Of course, this message had dozens of fields that all had to have

valid data in them—data to which the programmer had no access, since all the programmer had was a name, address, and phone number. After faking the data, the programmer had to jam the ID of the newly created contact into the sales record and send the `UpdateContact` message to the `SaleRecordService`.

Of course, to test anything you had to fire up all the necessary services, one by one, and fire up the message bus, and the BPel server, and ... And then, there were the propagation delays as these messages bounced from service to service, and waited in queue after queue.

And then if you wanted to add a new feature—well, you can imagine the coupling between all those services, and the sheer volume of WSDLs that needed changing, and all the redeployments those changes necessitated ...

Hell starts to seem like a nice place by comparison.

There's nothing intrinsically wrong with a software system that is structured around services. The error at W was the premature adoption and enforcement of a suite of tools that promised SoA—that is, the premature adoption of a massive suite of domain object services. The cost of those errors was sheer person-hours—person-hours in droves—flushed down the SoA vortex.

I could go on describing one architectural failure after another. But let's talk about an architectural success instead.

FITNESSE

My Son, Micah, and I started work on `FitNesse` in 2001. The idea was to create a simple wiki that wrapped Ward Cunningham's FIT tool for writing acceptance tests.

This was back in the days before Maven “solved” the jar file problem. I was adamant that anything we produced should not require people to download more than one jar file. I called this rule, “Download and Go.” This rule drove many of our decisions.

One of the first decisions was to write our own web server, specific to the needs of `FitNesse`. This might sound absurd. Even in 2001 there were plenty of open source web servers that we could have used. Yet writing our own turned out to be a really good decision because a bare-bones web server is a very simple piece of software to write and it allowed us to postpone any web framework decision until much later.²

Another early decision was to avoid thinking about a database. We had MySQL in the back of our minds, but we purposely delayed that decision by employing a design that made the decision irrelevant. That design was simply to put an interface between all data accesses and the data repository itself.

We put the data access methods into an interface named `WikiPage`. Those methods provided all the functionality we needed to find, fetch, and save pages. Of course, we didn't implement those methods at first; we simply stubbed them out while we worked on features that didn't involve fetching and saving the data.

Indeed, for three months we simply worked on translating wiki text into HTML. This didn't require any kind of data storage, so we created a class named `MockWikiPage` that simply left the data access methods stubbed.

Eventually, those stubs became insufficient for the features we wanted to write. We needed real data access, not stubs. So we created a new derivative of `WikiPage` named `InMemoryPage`. This derivative implemented the data access method to manage a hash table of wiki pages, which we kept in RAM.

This allowed us to write feature after feature for a full year. In fact, we got the whole first version of the `FitNesse` program working this way. We could create pages, link to other pages, do all the fancy wiki formatting, and even run tests with FIT. What we couldn't do was save any of our work.

When it came time to implement persistence, we thought again about MySQL, but decided that wasn't necessary in the short term, because it would be really easy to write the hash tables out to flat files. So we implemented `FileSystemWikiPage`, which just moved the functionality out to flat files, and then we continued developing more features.

Three months later, we reached the conclusion that the flat file solution was good enough; we decided to abandon the idea of MySQL altogether. We deferred that decision into nonexistence and never looked back.

That would be the end of the story if it weren't for one of our customers who decided that he needed to put the wiki into MySQL for his own purposes. We showed him the architecture of `WikiPages` that had allowed us to defer the decision. He came back *a day later* with the whole system working in MySQL. He simply wrote a `MySqlWikiPage` derivative and got it working.

We used to bundle that option with `FitNesse`, but nobody else ever used it, so

eventually we dropped it. Even the customer who wrote the derivative eventually dropped it.

Early in the development of `FitNesse`, we drew a *boundary line* between business rules and databases. That line prevented the business rules from knowing anything at all about the database, other than the simple data access methods. That decision allowed us to defer the choice and implementation of the database for well over a year. It allowed us to try the file system option, and it allowed us to change direction when we saw a better solution. Yet it did not prevent, or even impede, moving in the original direction (MySQL) when someone wanted it.

The fact that we did not have a database running for 18 months of development meant that, for 18 months, we did not have schema issues, query issues, database server issues, password issues, connection time issues, and all the other nasty issues that raise their ugly heads when you fire up a database. It also meant that all our tests ran fast, because there was no database to slow them down.

In short, drawing the boundary lines helped us delay and defer decisions, and it ultimately saved us an enormous amount of time and headaches. And that's what a good architecture should do.

WHICH LINES DO YOU DRAW, AND WHEN DO YOU DRAW THEM?

You draw lines between things that matter and things that don't. The GUI doesn't matter to the business rules, so there should be a line between them. The database doesn't matter to the GUI, so there should be a line between them. The database doesn't matter to the business rules, so there should be a line between them.

Some of you may have rejected one or more of those statements, especially the part about the business rules not caring about the database. Many of us have been taught to believe that the database is inextricably connected to the business rules. Some of us have even been convinced that the database is the embodiment of the business rules.

But, as we shall see in another chapter, this idea is misguided. The database is a tool that the business rules can use *indirectly*. The business rules don't need to know about the schema, or the query language, or any of the other details about the database. All the business rules need to know is that there is a set of functions that

can be used to fetch or save data. This allows us to put the database behind an interface.

You can see this clearly in [Figure 17.1](#). The `BusinessRules` use the `DatabaseInterface` to load and save data. The `DatabaseAccess` implements the interface and directs the operation of the actual `Database`.

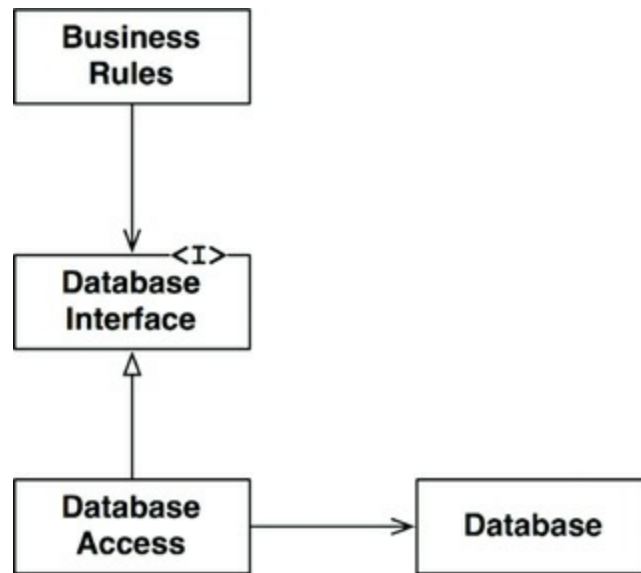


Figure 17.1 The database behind an interface

The classes and interfaces in this diagram are symbolic. In a real application, there would be many business rule classes, many database interface classes, and many database access implementations. All of them, though, would follow roughly the same pattern.

Where is the boundary line? The boundary is drawn across the inheritance relationship, just below the `DatabaseInterface` ([Figure 17.2](#)).

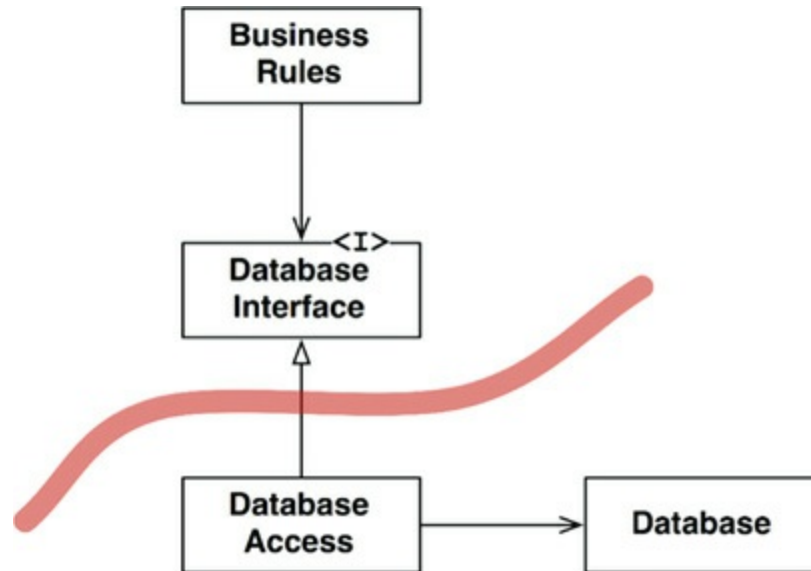


Figure 17.2 The boundary line

Note the two arrows leaving the `DatabaseAccess` class. Those two arrows point away from the `DatabaseAccess` class. That means that none of these classes knows that the `DatabaseAccess` class exists.

Now let's pull back a bit. We'll look at the component that contains many business rules, and the component that contains the database and all its access classes ([Figure 17.3](#)).

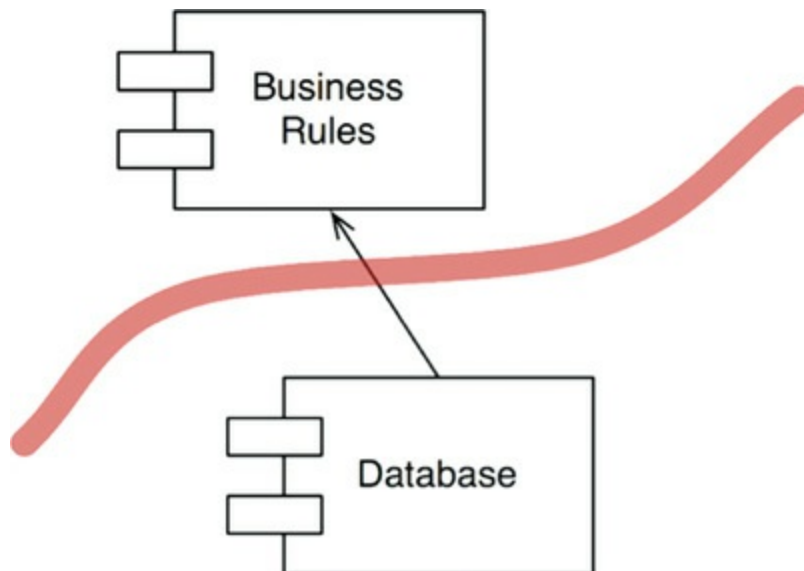


Figure 17.3 The business rules and database components

Note the direction of the arrow. The `Database` knows about the `BusinessRules`. The `BusinessRules` do not know about the `Database`. This implies that the

`DatabaseInterface` classes live in the `BusinessRules` component, while the `DatabaseAccess` classes live in the `Database` component.

The direction of this line is important. It shows that the `Database` does not matter to the `BusinessRules`, but the `Database` cannot exist without the `BusinessRules`.

If that seems strange to you, just remember this point: The `Database` component contains the code that translates the calls made by the `BusinessRules` into the query language of the database. It is that translation code that knows about the `BusinessRules`.

Having drawn this boundary line between the two components, and having set the direction of the arrow toward the `BusinessRules`, we can now see that the `BusinessRules` could use *any* kind of database. The `Database` component could be replaced with many different implementations—the `BusinessRules` don't care.

The database could be implemented with Oracle, or MySQL, or Couch, or Datomic, or even flat files. The business rules don't care at all. And that means that the database decision can be deferred and you can focus on getting the business rules written and tested before you have to make the database decision.

WHAT ABOUT INPUT AND OUTPUT?

Developers and customers often get confused about what the system is. They see the GUI, and think that the GUI is the system. They define a system in terms of the GUI, so they believe that they should see the GUI start working immediately. They fail to realize a critically important principle: *The IO is irrelevant*.

This may be hard to grasp at first. We often think about the behavior of the system in terms of the behavior of the IO. Consider a video game, for example. Your experience is dominated by the interface: the screen, the mouse, the buttons, and the sounds. You forget that behind that interface there is a model—a sophisticated set of data structures and functions—driving it. More importantly, that model does not need the interface. It would happily execute its duties, modeling all the events in the game, without the game ever being displayed on the screen. The interface does not matter to the model—the business rules.

And so, once again, we see the `GUI` and `BusinessRules` components separated by a boundary line ([Figure 17.4](#)). Once again, we see that the less relevant component

depends on the more relevant component. The arrows show which component knows about the other and, therefore, which component cares about the other. The `GUI` cares about the `BusinessRules`.

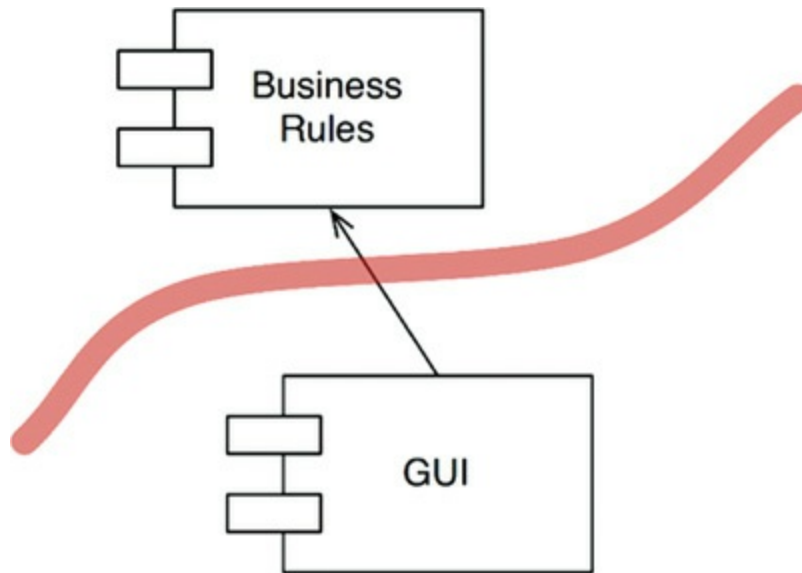


Figure 17.4 The boundary between `GUI` and `BusinessRules` components

Having drawn this boundary and this arrow, we can now see that the `GUI` could be replaced with any other kind of interface—and the `BusinessRules` would not care.

PLUGIN ARCHITECTURE

Taken together, these two decisions about the database and the `GUI` create a kind of pattern for the addition of other components. That pattern is the same pattern that is used by systems that allow third-party plugins.

Indeed, the history of software development technology is the story of how to conveniently create plugins to establish a scalable and maintainable system architecture. The core business rules are kept separate from, and independent of, those components that are either optional or that can be implemented in many different forms ([Figure 17.5](#)).

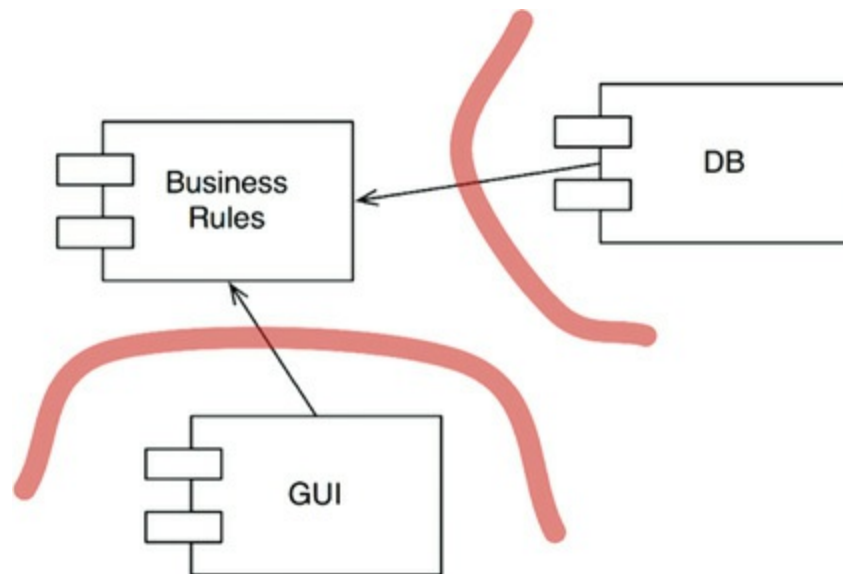


Figure 17.5 Plugging in to the business rules

Because the user interface in this design is considered to be a plugin, we have made it possible to plug in many different kinds of user interfaces. They could be web based, client/server based, SOA based, Console based, or based on any other kind of user interface technology.

The same is true of the database. Since we have chosen to treat it as a plugin, we can replace it with any of the various SQL databases, or a NOSQL database, or a file system-based database, or any other kind of database technology we might deem necessary in the future.

These replacements might not be trivial. If the initial deployment of our system was web-based, then writing the plugin for a client-server UI could be challenging. It is likely that some of the communications between the business rules and the new UI would have to be reworked. Even so, by starting with the presumption of a plugin structure, we have at very least made such a change practical.

THE PLUGIN ARGUMENT

Consider the relationship between ReSharper and Visual Studio. These components are produced by completely different development teams in completely different companies. Indeed, JetBrains, the maker of ReSharper, lives in Russia. Microsoft, of course, resides in Redmond, Washington. It's hard to imagine two development teams that are more separate.

Which team can damage the other? Which team is immune to the other? The dependency structure tells the story ([Figure 17.6](#)). The source code of ReSharper depends on the source code of Visual Studio. Thus there is nothing that the ReSharper team can do to disturb the Visual Studio team. But the Visual Studio team could completely disable the ReSharper team if they so desired.

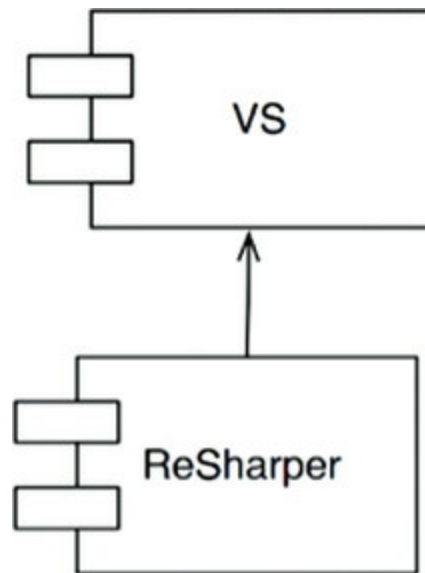


Figure 17.6 ReSharper depends on Visual Studio

That's a deeply asymmetric relationship, and it is one that we desire to have in our own systems. We want certain modules to be immune to others. For example, we don't want the business rules to break when someone changes the format of a web page, or changes the schema of the database. We don't want changes in one part of the system to cause other unrelated parts of the system to break. We don't want our systems to exhibit that kind of fragility.

Arranging our systems into a plugin architecture creates firewalls across which changes cannot propagate. If the GUI plugs in to the business rules, then changes in the GUI cannot affect those business rules.

Boundaries are drawn where there is an *axis of change*. The components on one side of the boundary change at different rates, and for different reasons, than the components on the other side of the boundary.

GUIs change at different times and at different rates than business rules, so there should be a boundary between them. Business rules change at different times and for different reasons than dependency injection frameworks, so there should be a boundary between them.

This is simply the Single Responsibility Principle again. The SRP tells us where to draw our boundaries.

CONCLUSION

To draw boundary lines in a software architecture, you first partition the system into components. Some of those components are core business rules; others are plugins that contain necessary functions that are not directly related to the core business. Then you arrange the code in those components such that the arrows between them point in one direction—toward the core business.

You should recognize this as an application of the Dependency Inversion Principle and the Stable Abstractions Principle. Dependency arrows are arranged to point from lower-level details to higher-level abstractions.

1. The word “architecture” appears in quotes here because three-tier is not an architecture; it’s a topology. It’s exactly the kind of decision that a good architecture strives to defer.
2. Many years later we were able to slip the Velocity framework into `FitNesse`.